✅ TASK: Share an EBS volume between two EC2 instances using NFS:

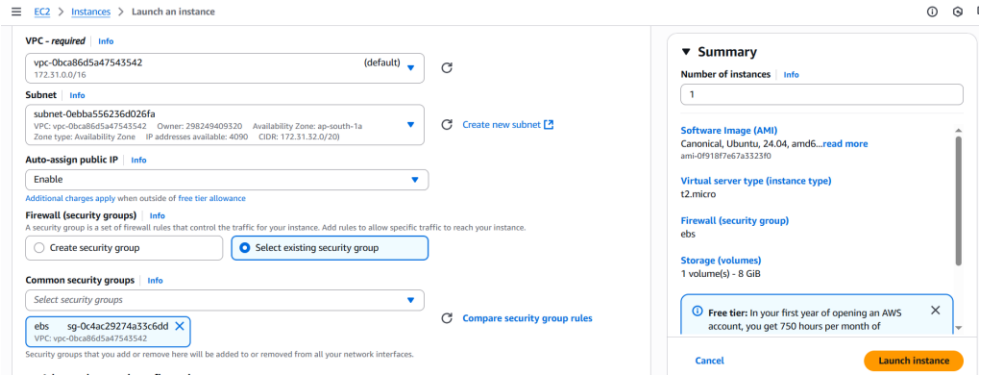
🧱 **STEP 1: Create Two EC2 Instances**

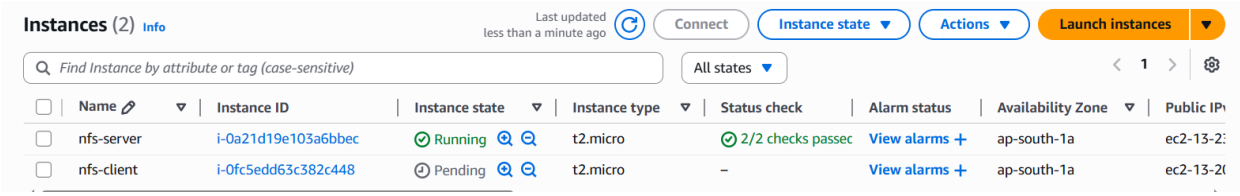
🔷 **Go to AWS Console → EC2 → Launch Instance**

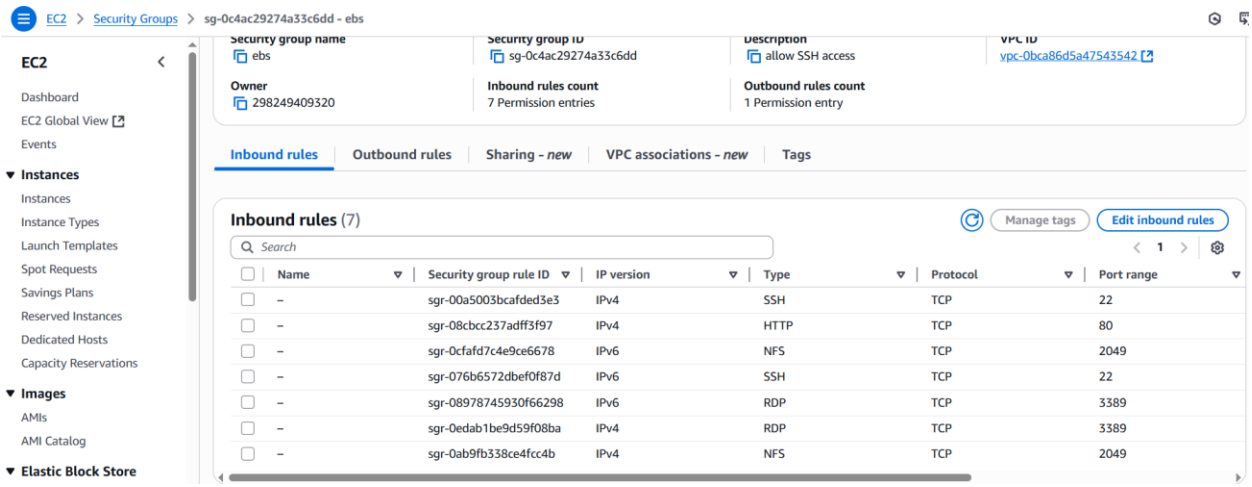
Repeat this **twice**, once for **Instance A** and once for **Instance B**.

🚀 **For both instances:**

1. **Name**:
   - Instance A: nfs-server
   - Instance B: nfs-client
2. **AMI**:
   - Choose **Ubuntu Server 22.04 LTS**
3. **Instance Type**:
   - Choose t2.micro (free tier eligible)
4. **Key Pair**:
   - Select or create a key pair (download .pem file)
5. **Network Settings**:
   - Place both instances in **the same VPC** and **same Availability Zone (AZ)** (e.g., us-east-1a)
6. **Security Group** (very important!):
   - Create or edit one with these **inbound rules**:
     - SSH (22) – your IP
     - NFS (2049) – allow from the **private IP or security group** of the other instance
7. **Launch the instance**



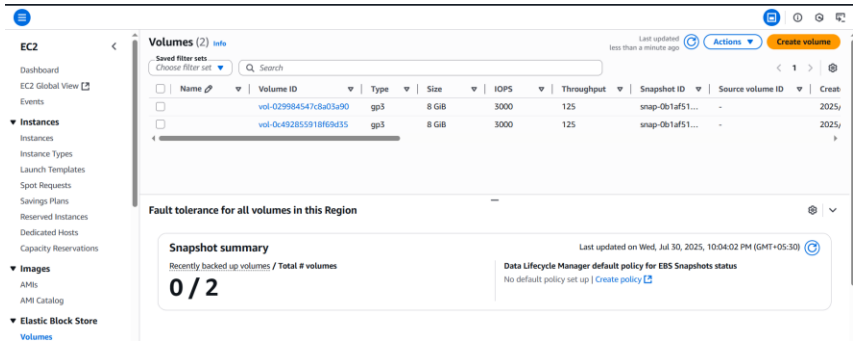


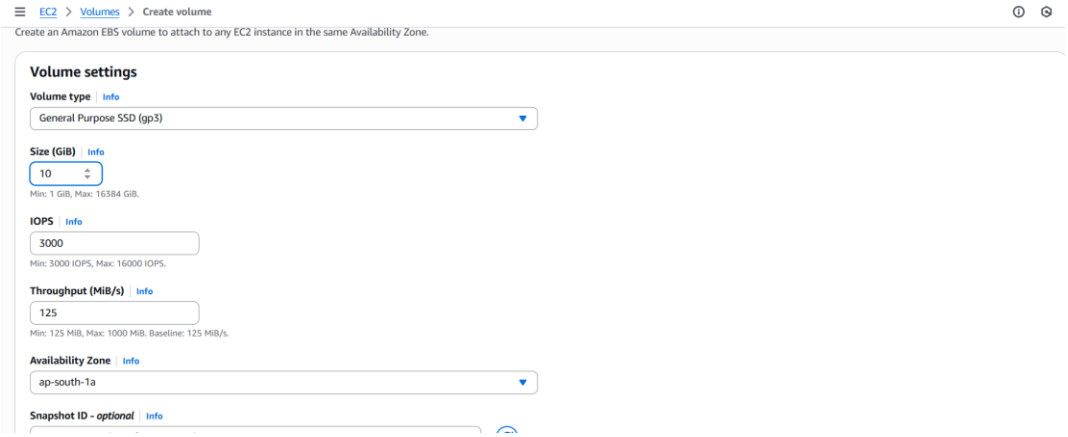


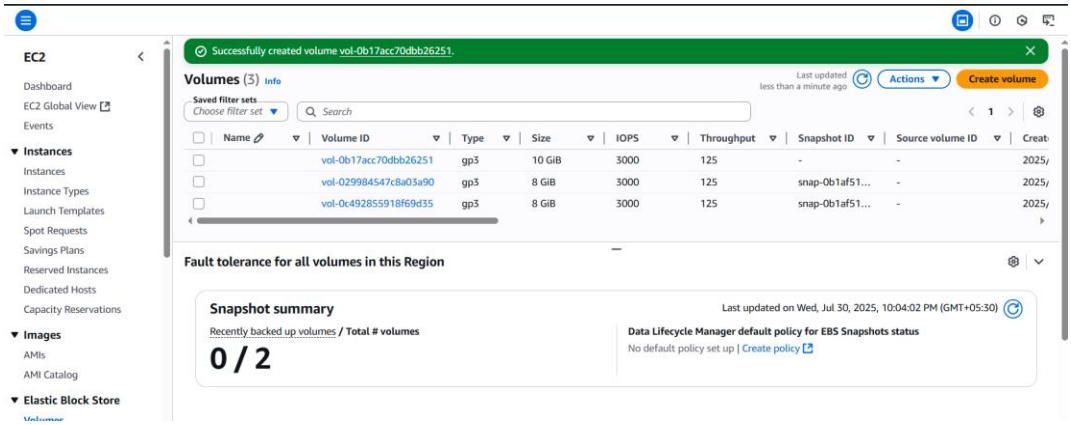
STEP 2: Create and Attach an EBS Volume to Instance A

Go to EC2 → Elastic Block Store → Volumes

1. Click **Create Volume**
2. Settings:
   - Size: 10 GiB
   - Volume Type: gp2
   - Availability Zone: Same as your EC2s (e.g., us-east-1a)
3. Click **Create Volume**

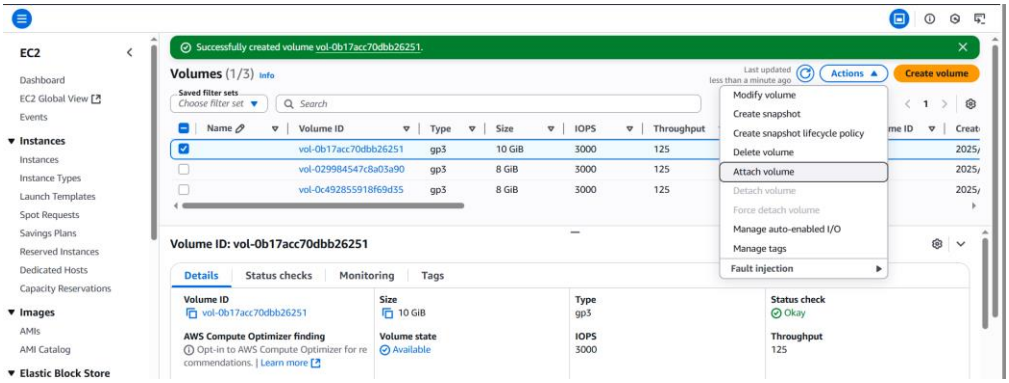


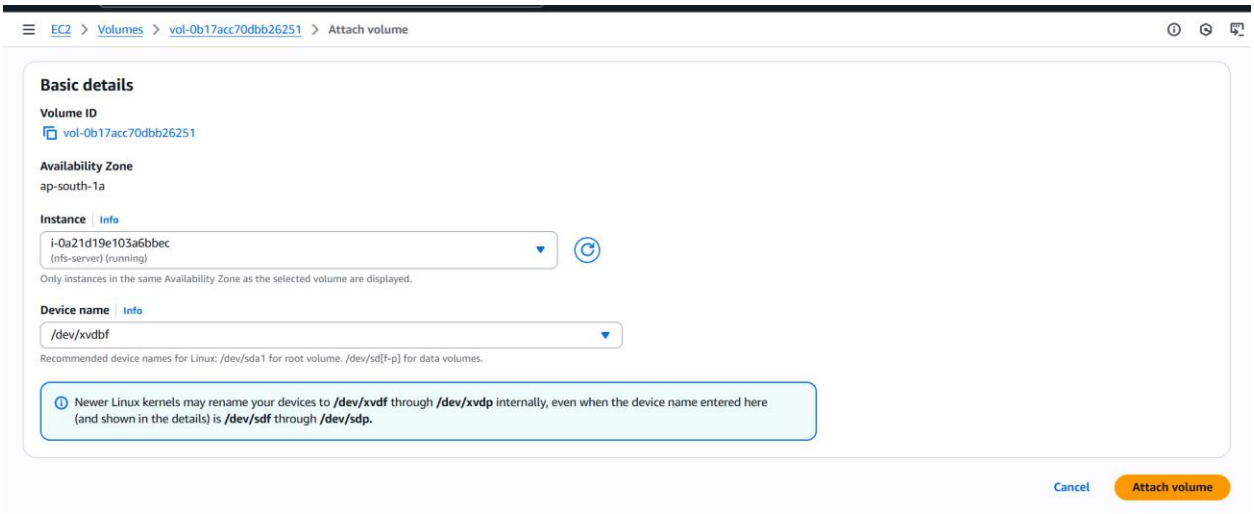


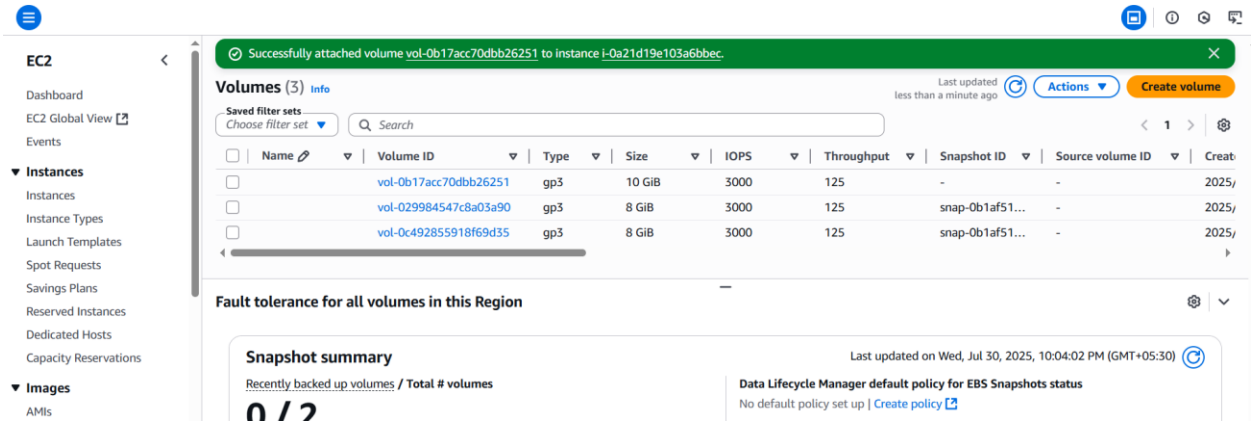


📎 Attach Volume to Instance A

1. Select the volume → Click **Actions → Attach Volume**
2. Choose nfs-server (Instance A)
3. Device name: /dev/xvdf
4. Click **Attach**







🔌 STEP 3: SSH into Both Instances

From your terminal (you'll need your .pem key file):

bash

CopyEdit

```
# Instance A (NFS server)
ssh -i your-key.pem ubuntu@<InstanceA-Public-IP>

# Instance B (NFS client)
ssh -i your-key.pem ubuntu@<InstanceB-Public-IP>
```

Replace <InstanceA-Public-IP> and <InstanceB-Public-IP> with actual public IPs from the EC2 dashboard.

```
ubuntu@ip-172-31-44-149: ~
Microsoft Windows [Version 10.0.19045.6093]
(c) Microsoft Corporation. All rights reserved.

C:\Users\WELCOME>cd Downloads

C:\Users\WELCOME\Downloads>ssh -i "ebs.pem" ubuntu@ec2-13-233-214-245.ap-south-1.compute.amazonaws.com
The authenticity of host 'ec2-13-233-214-245.ap-south-1.compute.amazonaws.com (64:ff9b::de9:d6f5)' can't be established.

ED25519 key fingerprint is SHA256:wGhQne4RZFgr2BrDVTABCsYvbOP9rI6YCmuDIcjLWGQ.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-13-233-214-245.ap-south-1.compute.amazonaws.com' (ED25519) to the list of known hosts.
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-1029-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Wed Jul 30 16:40:31 UTC 2025

  System load:  0.16              Processes:             104
  Usage of /:   25.3% of 6.71GB   Users logged in:       0
  Memory usage: 20%               IPv4 address for enX0: 172.31.44.149
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
```

```
ubuntu@ip-172-31-35-127: ~
  Swap usage:   0%

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status


The list of available updates is more than a week old.
To check for new updates run: sudo apt update


The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-35-127:~$
```

🏗️ STEP 4: Prepare EBS Volume on Instance A

On Instance A (nfs-server):

bash

CopyEdit

Format the disk

sudo mkfs.ext4 /dev/xvdbf

```
ubuntu@ip-172-31-44-149: ~
Get:44 http://security.ubuntu.com/ubuntu noble-security/universe amd64 Components [52.2 kB]
Get:45 http://security.ubuntu.com/ubuntu noble-security/universe amd64 c-n-f Metadata [17.0 kB]
Get:46 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Packages [1554 kB]
Get:47 http://security.ubuntu.com/ubuntu noble-security/restricted Translation-en [341 kB]
Get:48 http://security.ubuntu.com/ubuntu noble-security/restricted amd64 Components [212 B]
Get:49 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Packages [18.5 kB]
Get:50 http://security.ubuntu.com/ubuntu noble-security/multiverse Translation-en [4288 B]
Get:51 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 Components [208 B]
Get:52 http://security.ubuntu.com/ubuntu noble-security/multiverse amd64 c-n-f Metadata [380 B]
Fetched 36.0 MB in 20s (1843 kB/s)
Reading package lists... Done
ubuntu@ip-172-31-44-149:~$ sudo mkfs.ext4 /dev/xvdf
mke2fs 1.47.0 (5-Feb-2023)
The file /dev/xvdf does not exist and no size was specified.
ubuntu@ip-172-31-44-149:~$ sudo mkfs.ext4 /dev/xvbdf
mke2fs 1.47.0 (5-Feb-2023)
The file /dev/xvbdf does not exist and no size was specified.
ubuntu@ip-172-31-44-149:~$ sudo mkfs.ext4 /dev/xvdbf
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 2621440 4k blocks and 655360 inodes
Filesystem UUID: a4a9929f-1221-4207-a127-df814272b47d
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

ubuntu@ip-172-31-44-149:~$
```

Create a mount directory

sudo mkdir /mnt/ebs

Mount the volume

sudo mount /dev/xvdbf /mnt/ebs

Make it writable for testing

sudo chmod 777 /mnt/ebs

📡 STEP 5: Set Up NFS Server on Instance A

Still on Instance A:

bash

CopyEdit

Install NFS server

sudo apt update

sudo apt install nfs-kernel-server -y

```
ubuntu@ip-172-31-44-149: ~
mke2fs 1.47.0 (5-Feb-2023)
The file /dev/xvbdf does not exist and no size was specified.
ubuntu@ip-172-31-44-149:~$ sudo mkfs.ext4 /dev/xvdbf
mke2fs 1.47.0 (5-Feb-2023)
Creating filesystem with 2621440 4k blocks and 655360 inodes
Filesystem UUID: a4a9929f-1221-4207-a127-df814272b47d
Superblock backups stored on blocks:
        32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

ubuntu@ip-172-31-44-149:~$ sudo mkdir /mnt/ebs
ubuntu@ip-172-31-44-149:~$ sudo mount /dev/xvdbf /mnt/ebs
ubuntu@ip-172-31-44-149:~$ sudo chmod 777 /mnt/ebs
ubuntu@ip-172-31-44-149:~$ ls
ubuntu@ip-172-31-44-149:~$ sudo apt update
Hit:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble InRelease
Hit:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble-backports InRelease
Hit:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
98 packages can be upgraded. Run 'apt list --upgradable' to see them.
ubuntu@ip-172-31-44-149:~$ sudo apt install nfs-kernel-server -y
Reading package lists... Done
Building dependency tree... Done
```

Export the directory to Instance B

sudo nano /etc/exports

Add this line at the bottom (replace <Private-IP-B> with Instance B's **private IP**):

bash

CopyEdit

/data (rw,sync,no_subtree_check)

Save and exit (Ctrl+O, Enter, Ctrl+X)

bash

CopyEdit

Apply export rules

sudo exportfs -a

Restart NFS server

sudo systemctl restart nfs-kernel-server

```
ubuntu@ip-172-31-44-149: ~
nfs-mountd.service is a disabled or a static unit, not starting it.
nfsdcld.service is a disabled or a static unit, not starting it.

Creating config file /etc/exports with new version

Creating config file /etc/default/nfs-kernel-server with new version
Processing triggers for man-db (2.12.0-4build2) ...#############################################################.....]
Processing triggers for libc-bin (2.39-0ubuntu8.4) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-44-149:~$ sudo nano /etc/exports
ubuntu@ip-172-31-44-149:~$ sudo nano /etc/exports
ubuntu@ip-172-31-44-149:~$ sudo exportfs -a
ubuntu@ip-172-31-44-149:~$ sudo systemctl restart nfs-kernel-server
ubuntu@ip-172-31-44-149:~$
```

🔒 STEP 6: Check or Update Security Groups

In **AWS Console → EC2 → Security Groups**:

- Add **inbound rules** to allow NFS traffic (port 2049) from the other instance's **private IP or security group**

Inbound rules (7)

Manage tags | Edit inbound rules

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range
–	sgr-00a5003bcafded3e3	IPv4	SSH	TCP	22
–	sgr-08cbcc237adff3f97	IPv4	HTTP	TCP	80
–	sgr-0cfafd7c4e9ce6678	IPv6	NFS	TCP	2049
–	sgr-076b6572dbef0f87d	IPv6	SSH	TCP	22
–	sgr-08978745930f66298	IPv6	RDP	TCP	3389
–	sgr-0edab1be9d59f08ba	IPv4	RDP	TCP	3389
–	sgr-0ab9fb338ce4fcc4b	IPv4	NFS	TCP	2049

🔄 STEP 7: Mount the NFS Share on Instance B

On Instance B (nfs-client):

bash

CopyEdit

Install NFS client

sudo apt update

sudo apt install nfs-common -y

Create mount point

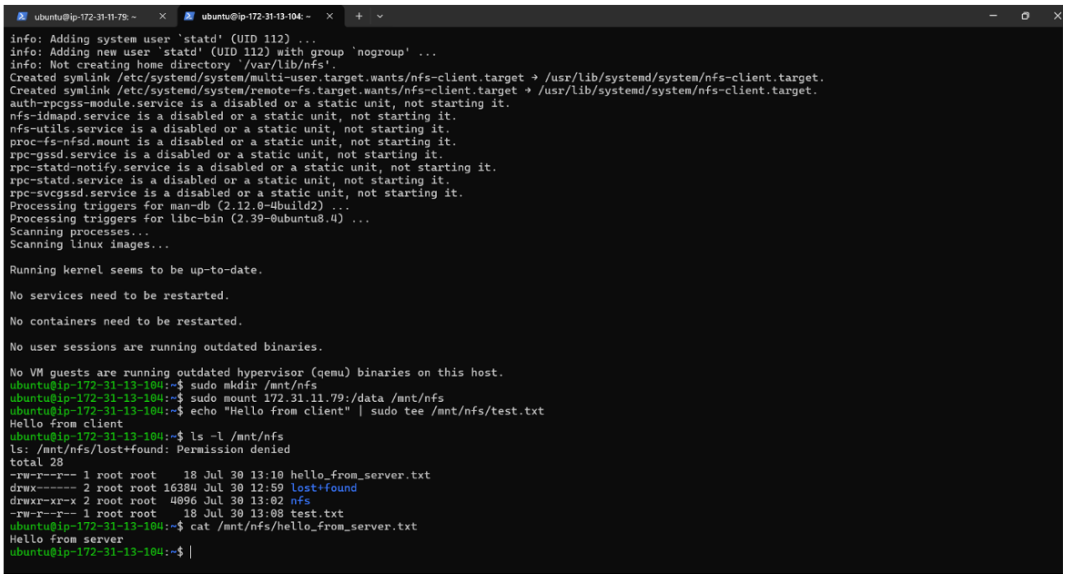
sudo mkdir /mnt/nfs

Mount NFS share (replace <Private-IP-A>)

sudo mount 172.31.44.149:/mnt/ebs /mnt/nfs

Make it writable for testing

sudo chmod 777 /mnt/nfs



✅ STEP 8: Test the Setup

On Instance B:

bash

CopyEdit

echo "Hello from Instance B" | sudo tee /mnt/nfs/test.txt

On Instance A:

bash

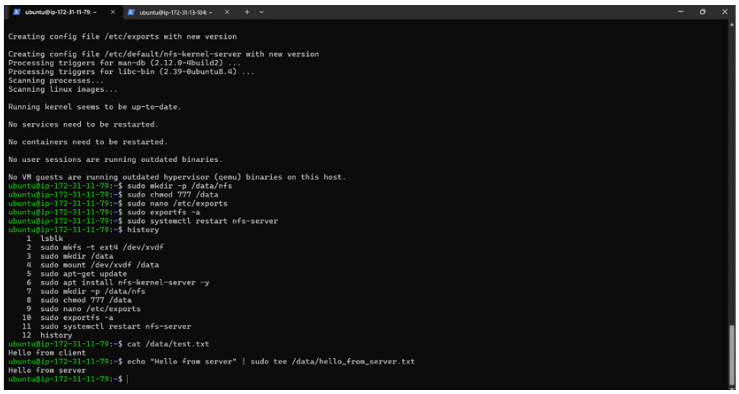
CopyEdit

cat /mnt/ebs/test.txt

You should see the message:

Hello from Instance B

🎉 It works!



📊 BONUS: Compare with EFS (Optional)

1. Go to AWS → **EFS → Create File System**
2. Follow defaults (same VPC and AZ)
3. Add a **mount target** for the subnet

On both instances:

bash

CopyEdit

sudo apt install -y amazon-efs-utils

sudo mkdir /mnt/efs

sudo mount -t efs <fs-ID>:/ /mnt/efs

Test writing to /mnt/efs from either instance.

Feature	EBS (via NFS)	EFS
Type	Block Storage (shared via NFS manually)	Native Network File System
Access	Single EC2 instance (NFS server) shares storage	Concurrent access from multiple EC2s
Protocol	NFS (custom-configured)	NFSv4.1 / NFSv4.0 (fully managed)
Durability & Availability	Depends on EC2 instance & setup	Highly available, across multiple AZs
Elasticity	Manual scaling	Automatic scaling

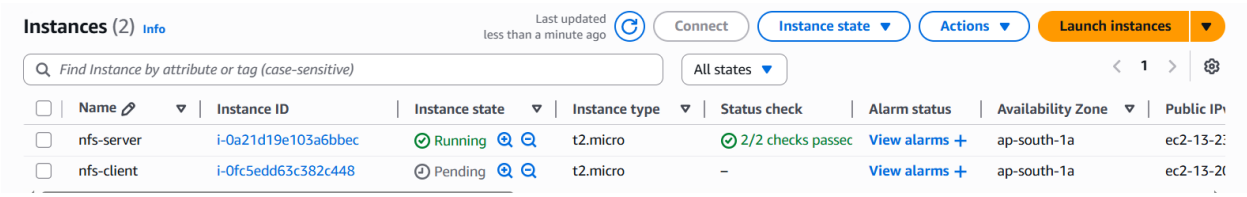
Task 2: Set up EFS and Mount on Linux & Windows Instances with Concurrent Access Testing:

Step 1: Launch EC2 Instances (Linux and Windows)

🔷 1.1 Create a Linux EC2 Instance (Amazon Linux 2)

✅ In AWS Console:

1. Go to EC2 → Instances → Launch Instance
2. Name: efs-linux
3. AMI: Amazon Linux 2 (or Ubuntu if preferred)
4. Instance Type: t2.micro or larger (for test purposes)
5. Key Pair: Choose or create one (for SSH access)
6. Network Settings:
   - VPC: Same as EFS
   - Subnet: Choose an AZ that supports EFS mount targets
   - Auto-assign Public IP: Enable
   - Create or select a Security Group:
     - Allow SSH (port 22)
     - Allow NFS (TCP 2049) from Windows SG or 0.0.0.0/0 for testing
     - Allow SMB (TCP 445) from Windows EC2
7. Click Launch Instance



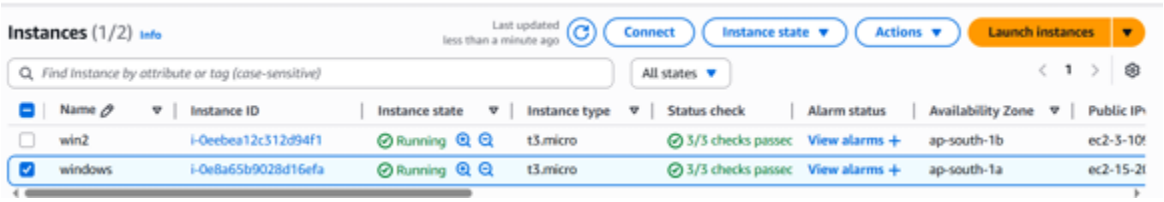
🔷 1.2 Create a Windows EC2 Instance

✅ In AWS Console:

1. Name: efs-windows
2. AMI: Microsoft Windows Server 2019 or 2022 Base
3. Instance Type: t2.medium or larger
4. Key Pair: Choose or create one (for RDP access)
5. Network Settings:

- Same VPC and AZ as Linux EC2
- Assign Public IP
- Security Group:
  - Allow RDP (port 3389)
  - Allow SMB (port 445) to Linux EC2

6. Click Launch Instance



📎 1.3 Security Groups Summary

Rule	Linux EC2	Windows EC2
SSH (22)	✅	❌
RDP (3389)	❌	✅
NFS (2049)	✅	❌
SMB (445)	✅ (from Windows SG)	❌

🔐 1.4 Connect to Instances

- Linux EC2: Use SSH

bash

CopyEdit

ssh -i your-key.pem ec2-user@<linux-public-ip>

- Windows EC2:
  - Get Windows Administrator password (via EC2 → Connect)
  - Use RDP client to connect with username Administrator

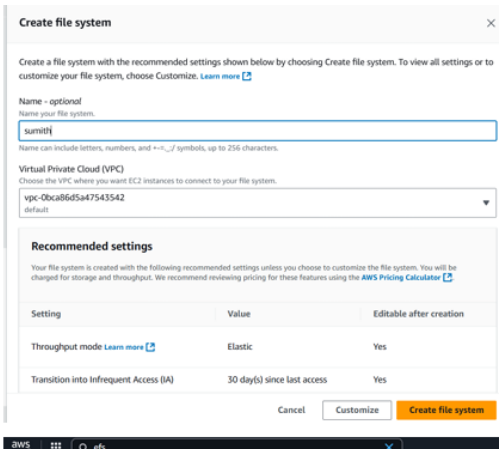
✅ At this point, you have:

- A Linux EC2 ready to mount EFS and share it via Samba
- A Windows EC2 ready to connect to the Samba share

Step 2: Create and Configure EFS

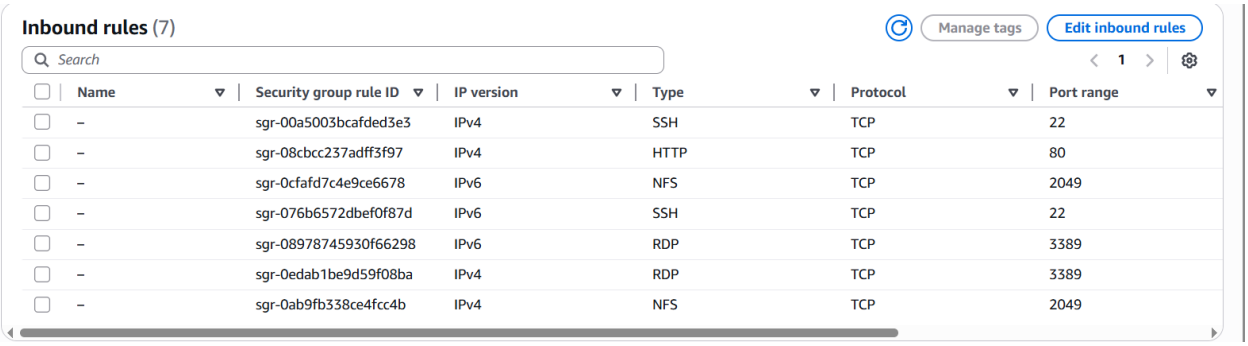
2.1 Create EFS File System

- Go to **EFS → Create File System**
- **Name**: shared-efs
- **VPC**: Same as EC2s
- Enable **automatic mount targets**
- Use default performance/burst settings



2.2 Set Up EFS Security

- Ensure EFS uses a security group allowing:
  - **TCP 2049 (NFS)** from Linux EC2
  - **TCP 2049 or 445** if planning alternative access



Inbound rules (7)

Manage tags | Edit inbound rules

Search

Name	Security group rule ID	IP version	Type	Protocol	Port range
–	sgr-00a5003bcafded3e3	IPv4	SSH	TCP	22
–	sgr-08cbcc237adff3f97	IPv4	HTTP	TCP	80
–	sgr-0cfafd7c4e9ce6678	IPv6	NFS	TCP	2049
–	sgr-076b6572dbef0f87d	IPv6	SSH	TCP	22
–	sgr-08978745930f66298	IPv6	RDP	TCP	3389
–	sgr-0edab1be9d59f08ba	IPv4	RDP	TCP	3389
–	sgr-0ab9fb338ce4fcc4b	IPv4	NFS	TCP	2049

🔷 **Step 3: Mount EFS on Linux EC2**

3.1 SSH into Linux EC2

bash

CopyEdit

```
ssh -i your-key.pem ec2-user@<Linux-EC2-IP>
```

3.2 Install Required Packages

bash

CopyEdit

```
sudo yum update -y
sudo yum install -y nfs-utils samba samba-client
```

3.3 Create Mount Point and Mount EFS

bash

CopyEdit

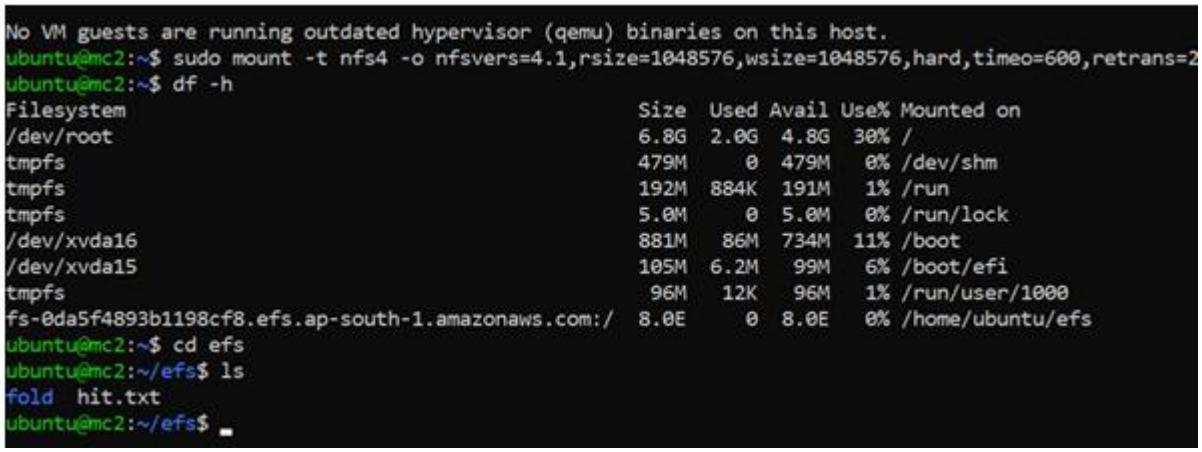
```
sudo mkdir -p /mnt/efs
sudo mount -t nfs4 -o nfsvers=4.1 fs-<id>.efs.<region>.amazonaws.com:/ /mnt/efs
```

3.4 Verify Mount

bash

CopyEdit

```
df -h | grep efs
```



```
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@mc2:~$ sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2
ubuntu@mc2:~$ df -h
Filesystem                                          Size  Used Avail Use% Mounted on
/dev/root                                           6.8G  2.0G  4.8G  30% /
tmpfs                                               479M     0  479M   0% /dev/shm
tmpfs                                               192M  884K  191M   1% /run
tmpfs                                               5.0M     0  5.0M   0% /run/lock
/dev/xvda16                                         881M   86M  734M  11% /boot
/dev/xvda15                                         105M  6.2M   99M   6% /boot/efi
tmpfs                                                96M   12K   96M   1% /run/user/1000
fs-0da5f4893b1198cf8.efs.ap-south-1.amazonaws.com:/  8.0E     0  8.0E   0% /home/ubuntu/efs
ubuntu@mc2:~$ cd efs
ubuntu@mc2:~/efs$ ls
fold  hit.txt
ubuntu@mc2:~/efs$
```

🔷 **Step 4: Configure Samba on Linux**

4.1 Prepare Share Directory

bash

CopyEdit

```
sudo mkdir -p /mnt/efs/shared
sudo chmod -R 777 /mnt/efs/shared
```

4.2 Edit Samba Configuration

bash

CopyEdit

```
sudo nano /etc/samba/smb.conf
```

Add at bottom:

ini

CopyEdit

```
[efs-share]
  path = /mnt/efs/shared
  read only = no
  browsable = yes
  guest ok = yes
  force user = ec2-user
```

4.3 Start and Enable Samba

bash

CopyEdit

```
sudo systemctl enable smb
sudo systemctl start smb
```

4.4 Open Firewall (if enabled)

bash

CopyEdit

```
sudo firewall-cmd --add-service=samba --permanent
sudo firewall-cmd –reload
```

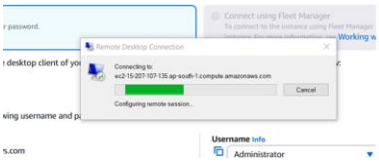
```
No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-11-147:~$ sudo mkdir /mnt/efs
ubuntu@ip-172-31-11-147:~$ sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvport fs-0863063a5581ff84c.efs.ap-sou
th-1.amazonaws.com:/ efs
mount.nfs4: mount point efs does not exist
ubuntu@ip-172-31-11-147:~$ sudo mount -t nfs4 -o nfsvers=4.1,rsize=1048576,wsize=1048576,hard,timeo=600,retrans=2,noresvport fs-0863063a5581ff84c.efs.ap-sou
th-1.amazonaws.com:/ /mnt/efs
ubuntu@ip-172-31-11-147:~$ df -h | grep efs
fs-0863063a5581ff84c.efs.ap-south-1.amazonaws.com:/  8.0E     0  8.0E   0% /mnt/efs
ubuntu@ip-172-31-11-147:~$ ls /mnt/efs
ubuntu@ip-172-31-11-147:~$ sudo apt install samba -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  attr libavahi-client3 libavahi-common-data libavahi-common3 libboost-iostreams1.83.0 libboost-thread1.83.0
  libcephfs2 libcups2t64 libldb2 librados2 librdmacm1t64 libtalloc2 libtdb1 libtevent0t64 liburing2 libwbclient0
  python3-dnspython python3-gpg python3-ldb python3-markdown python3-samba python3-talloc python3-tdb
  samba-ad-provision samba-common samba-common-bin samba-dsdb-modules samba-libs samba-vfs-modules tdb-tools
Suggested packages:
  cups-common python3-trio python3-aioquic python3-h2 python3-httpx python3-httpcore python-markdown-doc bind9
  bind9utils ctdb ldb-tools winbind heimdal-clients
The following NEW packages will be installed:
  attr libavahi-client3 libavahi-common-data libavahi-common3 libboost-iostreams1.83.0 libboost-thread1.83.0
  libcephfs2 libcups2t64 libldb2 librados2 librdmacm1t64 libtalloc2 libtdb1 libtevent0t64 liburing2 libwbclient0
  python3-dnspython python3-gpg python3-ldb python3-markdown python3-samba python3-talloc python3-tdb samba
  samba-ad-provision samba-common samba-common-bin samba-dsdb-modules samba-libs samba-vfs-modules tdb-tools
0 upgraded, 31 newly installed, 0 to remove and 92 not upgraded.
Need to get 19.3 MB of archives.
After this operation, 102 MB of additional disk space will be used.
Get:1 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 python3-dnspython all 2.6.1-1ubuntu1 [163 kB]
Get:2 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libtalloc2 amd64 2.4.2-1build2 [27.3 kB]
Get:3 http://ap-south-1.ec2.archive.ubuntu.com/ubuntu noble/main amd64 libtdb1 amd64 1.4.10-1build1 [46.8 kB]
```

```
ubuntu@ip-172-31-11-147: ~
Created symlink /etc/systemd/system/multi-user.target.wants/samba-ad-dc.service → /usr/lib/systemd/system/samba-ad-dc.service.
Processing triggers for ufw (0.36.2-6) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.4) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-11-147:~$ sudo nano /etc/samba/smb.conf
ubuntu@ip-172-31-11-147:~$ sudo smbpasswd -a ubuntu
New SMB password:
Retype new SMB password:
Added user ubuntu.
ubuntu@ip-172-31-11-147:~$ sudo systemctl restart smbd
ubuntu@ip-172-31-11-147:~$ sudo ufw allow 'Samba'
Rules updated
Rules updated (v6)
ubuntu@ip-172-31-11-147:~$ hostname -I
172.31.11.147
ubuntu@ip-172-31-11-147:~$ echo "Hello from Ubuntu EFS!" | sudo tee /mnt/efs/testfile.txt
Hello from Ubuntu EFS!
ubuntu@ip-172-31-11-147:~$ |
```

🔷 Step 5: Connect from Windows EC2

5.1 Use RDP to Login

- From AWS Console → Connect → Get Windows password
- Use Administrator + decrypted password
- RDP into instance using public IP



5.2 Access the Samba Share

Open **File Explorer** and type:

php-template

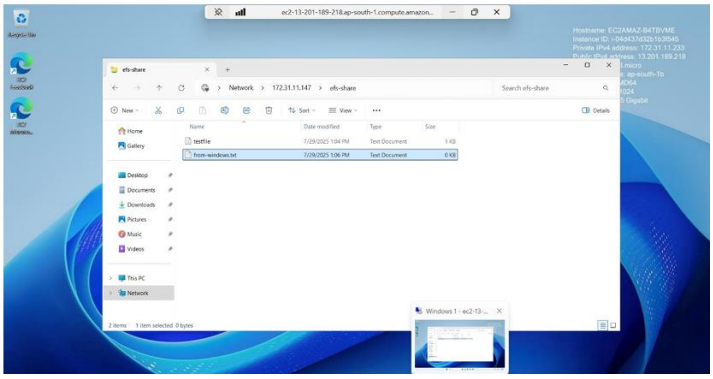
CopyEdit

\\<Linux-EC2-IP>\efs-share

If prompted for credentials:

- Username: guest
- Password: (leave blank)

You should now see the shared folder backed by EFS.



◆ Step 6: Test Concurrent File Access

6.1 On Linux:

bash

CopyEdit

echo "Hello from Linux" > /mnt/efs/shared/test.txt

6.2 On Windows:

- Open test.txt using Notepad
- Add: Hello from Windows
- Save

6.3 Back on Linux:

bash

CopyEdit

cat /mnt/efs/shared/test.txt

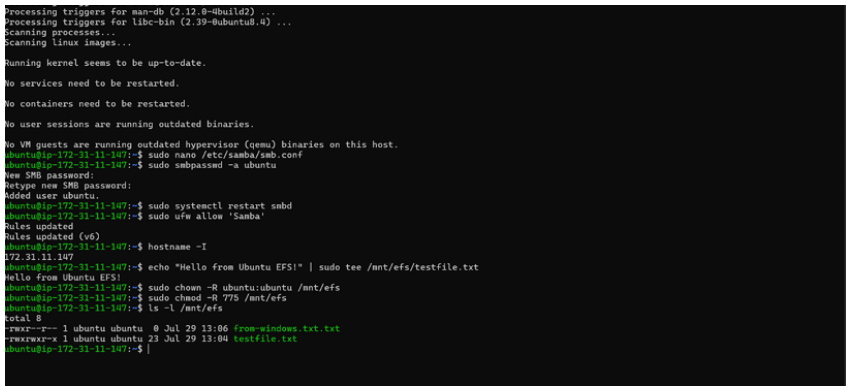
Expected output:

csharp

CopyEdit

Hello from Linux

Hello from Windows



✅ Success! Both systems are accessing the same EFS-backed file concurrently.

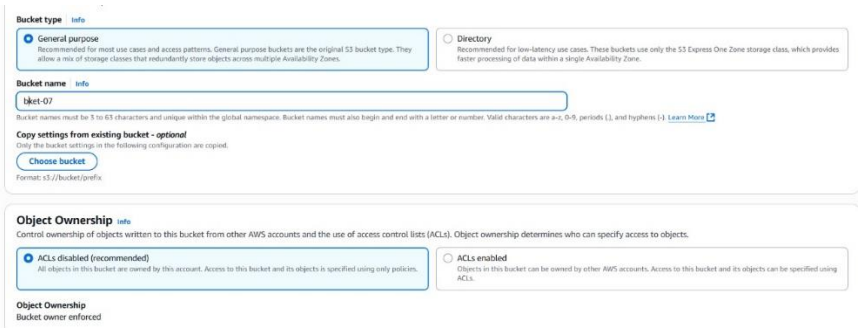
Configure S3 Lifecycle Policies for Glacier / Deep Archive

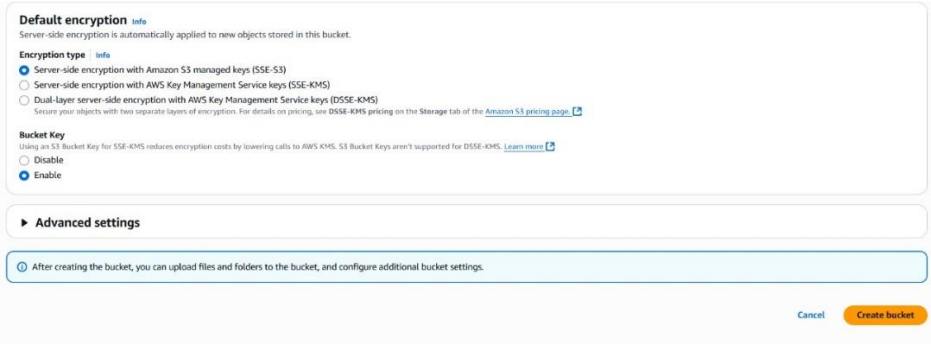
✅ Objective:

Automatically move S3 objects to **Glacier**, **Glacier Deep Archive**, or **delete them** after a set period.

🔷 Step 1: Create or Select an S3 Bucket

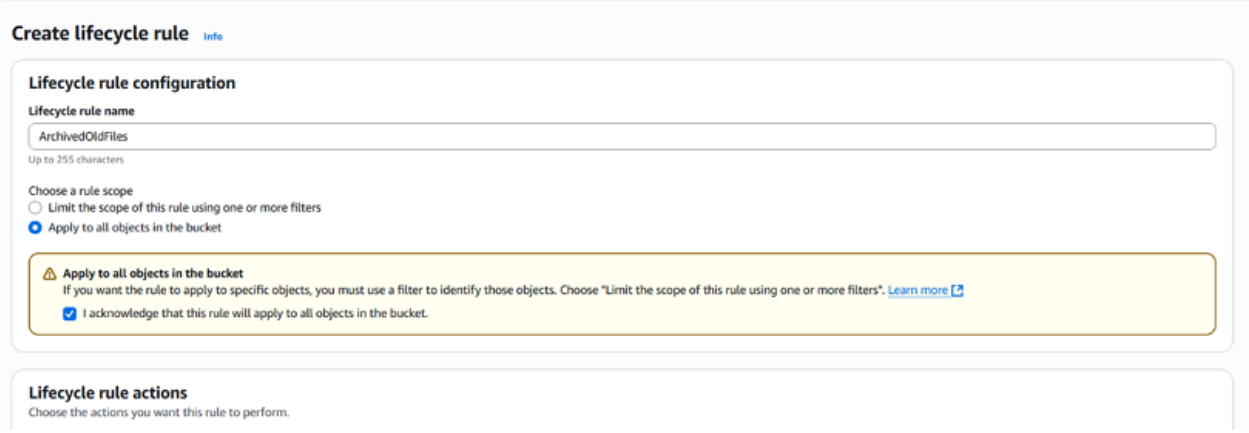
1. Go to **S3 Console**: https://s3.console.aws.amazon.com/s3/
2. Click **"Create bucket"** or select an existing one
3. Ensure versioning is **enabled** (required for some lifecycle options)





◆ Step 2: Add a Lifecycle Rule

1. In your bucket, go to the **"Management" tab**
2. Scroll to **“Lifecycle rules”**
3. Click **“Create lifecycle rule”**



🔷 Step 3: Configure Lifecycle Rule

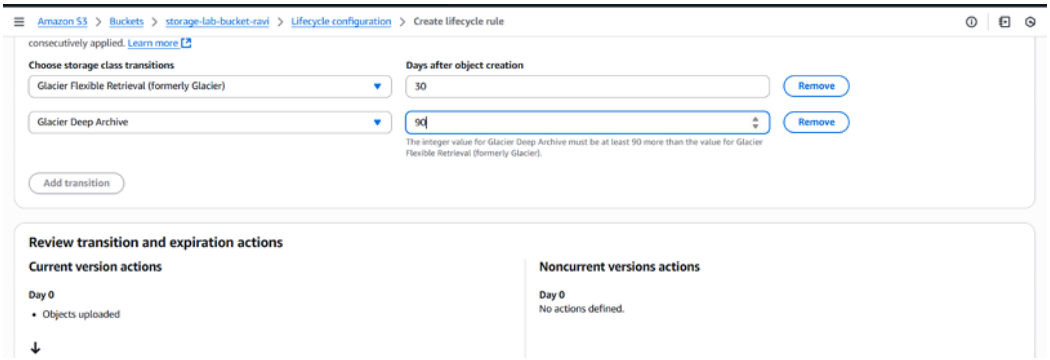
🔶 3.1 Rule Name

- Enter a rule name, e.g., transition-to-glacier

🔶 3.2 Rule Scope

Choose:

- ✅ **This rule applies to all objects in the bucket**, or
- 🔍 **Limit by prefix or tag** if applying to a specific folder or file type



🔷 Step 4: Define Transitions to Glacier / Deep Archive

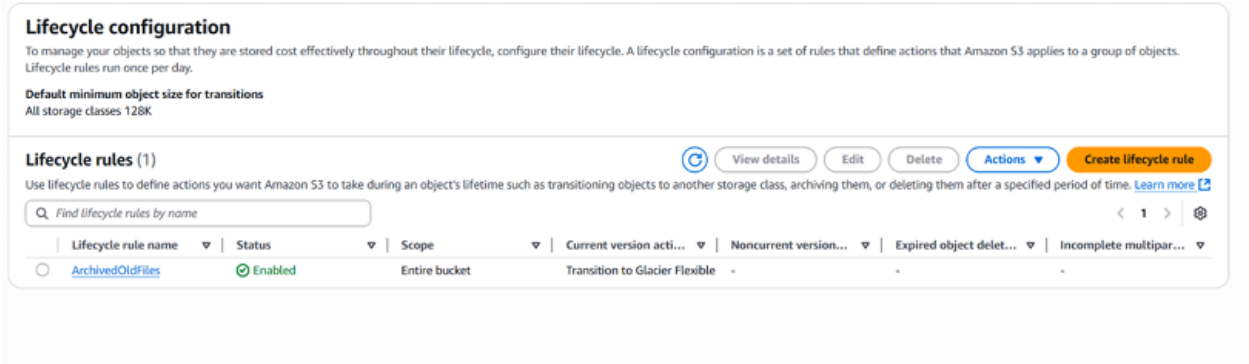
🔶 4.1 Add Transition Action

Click **"Add transition"**, then:

Transition Action	Days After Creation	Storage Class
Move to Glacier	e.g., 30 days	GLACIER (Instant Retrieval or Flexible Retrieval)
Move to Deep Archive	e.g., 180 days	DEEP_ARCHIVE

Example:

- Transition to **GLACIER Flexible Retrieval** after 30 days
- Transition to **DEEP_ARCHIVE** after 180 days (optional)



🔷 Step 5: (Optional) Add Expiration

You can also set:

- **Delete objects** after x days
- **Delete incomplete multipart uploads**

Example:

plaintext

CopyEdit

Expire objects 365 days after creation

🔷 Step 6: Review and Save

- Review the rule summary
- Click **"Create rule"**

✅ Your lifecycle rule is now active!

📄 **Example Policy Summary**

This lifecycle rule applies to **all objects**:

- Transition to **GLACIER Flexible Retrieval** after **30 days**
- Transition to **DEEP_ARCHIVE** after **180 days**
- **Delete** objects after **365 days** (optional)

⚠️ **Notes**

- **Transitioning costs money**, so avoid frequent transitions.
- **Glacier Instant Retrieval** is more expensive but provides millisecond access.
- **Glacier Deep Archive** has the lowest storage cost but 12–48 hours retrieval time.
- If you use **Intelligent Tiering**, AWS manages transitions automatically.

Scenario 1: Simulate an EBS Multi-Attach Failure

🎯 **Objective:**

Try to **attach a non-io1/io2 EBS volume** (e.g., gp2 or gp3) to **multiple EC2 instances** and **observe the failure**, then debug the issue.

🧪 **Step 1: Create a Non-Multi-Attach Volume**

1. Go to **EC2 → Elastic Block Store → Volumes**
2. Click **"Create Volume"**
   - **Volume type**: gp3 (General Purpose SSD)
   - **Size**: e.g., 8 GiB
   - **Availability Zone**: us-east-1a (or your AZ)
   - **Multi-Attach**: *Not available for this type*
3. Click **"Create Volume"**

🖥️ **Step 2: Launch Two EC2 Instances in Same AZ**

- Create **two EC2 instances** (Instance A and Instance B) in the same **Availability Zone** as the volume.
- Type: t2.micro or larger

- OS: Amazon Linux 2 (or Ubuntu)
- Enable SSH access

🔄 **Step 3: Attach the Volume to Instance A**

1. Go to **Volumes → Actions → Attach Volume**
2. Select **Instance A**
3. Device name: /dev/xvdf
4. Click **Attach**

   ✅ Success — Volume is now attached to Instance A.

🛑 **Step 4: Attempt to Attach Same Volume to Instance B**

1. Go to **Volumes → Actions → Attach Volume**
2. Select **Instance B**
3. Click **Attach**

   ❌ **You'll Get an Error:**

   Error Message:

   "The volume 'vol-xxxx' does not support Multi-Attach and is already attached to another instance."

🧵 **Step 5: Debug the Failure**

🔍 **Identify the Root Cause**

1. **Check Volume Type**:

   bash

   CopyEdit

   ```
   aws ec2 describe-volumes --volume-ids vol-xxxx --query 'Volumes[*].VolumeType'
   ```

   Output:

   arduino

   CopyEdit

   ```
   [
     "gp3"
   ]
   ```

   gp3 **does not support Multi-Attach**

2. **Check EC2 Console → Volume Attachment State**
   - Shows it's **in-use** on Instance A
   - Cannot attach again unless you **detach first**
3. **Check AWS Documentation:**
   - Only **io1** and **io2** volumes support **Multi-Attach**
   - Must be created with Multi-Attach enabled

✅ Step 6: Resolution

Option 1: Use a Compatible Volume

1. Create a new EBS volume:
   - **Type**: io1 or io2
   - **Enable Multi-Attach** (checkbox)
2. Attach it to **both EC2 instances** in the same AZ

Option 2: Change Architecture

If you need shared access but can't afford io1/io2:

- Use **EFS** for file-level sharing
- Use **NFS server** on one EC2 with EBS behind it

⚠️ Key Limitations of Multi-Attach

Constraint	Description
🔷 **Volume Type**	**Only io1 or io2 support Multi-Attach**
🔷 **Same AZ**	**All EC2s must be in the same Availability Zone**
🔷 **File System**	**Use cluster-aware file systems (e.g., GFS2) to avoid corruption**
🔷 **Max Instances**	**A volume can be attached to up to 16 instances simultaneously**

🧰 **Summary: Debugging Steps**

Step	Action
1	Try to attach a non-multi-attach volume (e.g., gp3) to 2 EC2s
2	Observe failure: attachment blocked
3	Use CLI or Console to check volume type
4	Consult AWS docs: only io1/io2 support it
5	Resolve: use io1/io2 or redesign architecture

Task: Configure Amazon EFS and Mount on Windows EC2 using NFSv4.1

🔧 **Task Summary**

Goal:

Set up Amazon EFS (Elastic File System) and mount it directly on a **Windows Server EC2 instance** using **NFSv4.1 protocol**.

📌 **Problem: Windows Does Not Support NFSv4.1**

⚠️ **Challenge 1: NFSv4.1 Not Supported on Windows**

- **Windows Server** only supports **NFSv2** and **NFSv3** (optional components), not **NFSv4.1**.
- **Amazon EFS only supports NFSv4.1 and NFSv4.0**.
- 🔴 **Result**: Direct mounting of EFS via NFS on Windows fails.

💡 **Workaround**: You must use a **Linux bridge server** (with Samba) or switch to **Amazon FSx for Windows** for native Windows SMB support.

🚫 **Common Network Error Messages**

When trying to mount EFS on Windows using NFS:

vbnet

CopyEdit

Error: The network path was not found.

or

Error: Mount failed with error 53 or error 1231.

🔍 **Reasons:**

- Windows cannot interpret the NFSv4.1 protocol used by EFS.
- Security group or firewall blocks port 2049 (NFS).
- Misunderstanding between **EFS vs. EBS shared via NFS** (see below).

🔄 **Clarifying Concepts: EFS vs. EBS via NFS**

Feature	EFS	EBS (via NFS)
Storage Type	Network File System (NFSv4.1)	Block Storage (shared by user-configured NFS)
Protocol	NFSv4.1 only	You choose (can set up NFSv3 or SMB via Linux)
Windows Compatibility	❌ Not supported directly	✅ Possible using Samba/NFS server on Linux
Management	AWS-managed	Fully self-managed
Availability Zones	Multi-AZ	Single AZ unless HA configured
Use Case	Shared file storage across many instances	High-performance disk with manual sharing

---

🛠️ **Resolution Options**

✅ **Option 1: Use a Linux EC2 as a Bridge (Samba Share)**

1. **Mount EFS on a Linux EC2 instance**
2. **Install and configure Samba**
3. **Share EFS over SMB** to the Windows instance
4. Windows can mount the share using \\<linux-ip>\efs-share

➡️ [Click here for full Samba-on-Linux guide]

✅ Option 2: Use Amazon FSx for Windows File Server

- Native **SMB protocol**, designed for Windows
- Can be joined to your Active Directory
- Works directly with **Windows File Explorer**
- Auto-backups, HA, and replication built-in

Use if:

- You **don't want to manage a Linux bridge**
- You need **enterprise-grade Windows file system**

📌 Best Practices

- **EFS is ideal** for Linux-based shared workloads
- **Windows workloads** should use **FSx for Windows**, or EFS via Samba bridge
- If using Samba, ensure:
  - Linux server is in same VPC and subnet
  - Security group allows **port 445** from Windows
  - EFS mount on Linux uses nfsvers=4.1

🧪 Optional Diagnostic Checklist

Check	Result
Port 2049 open to Linux EC2?	✅
Port 445 open to Windows EC2?	✅
Linux EC2 can mount EFS via NFSv4.1?	✅
Windows EC2 can access Samba share via \\ip\share?	✅
NFS service installed on Windows?	⚠️ (does not help for v4.1)

📌 **Summary**

Issue	Resolution
Windows cannot use EFS directly via NFSv4.1	Use **Samba** or switch to **FSx for Windows**
Network errors due to incompatible protocol	Confirm protocol and ports, use supported option
Confusion between EBS and EFS	EBS = block, EFS = file; use accordingly